

Week 1: Context Free Grammars

We start with some revision of sets. If you are unfamiliar with any of the notation, please ask one of the TAs.

- * 1. Write $\mathbb{N}$ for the set of natural numbers $0, 1, 2, \dots$, and write Σ for the alphabet $\{0, 1\}$. List the elements of the following sets in any order.

- (a) $\{1, 2, 3\} \cup \{2, 4, 6\}$
- (b) $\{1, 2, 3\} \cap \{2, 4, 6\}$
- (c) $\{1, 2, 3\} \times \{2, 4, 6\}$
- (d) $\{1, 2, 3\} \setminus \{2, 4, 6\}$
- (e) $\{2m \mid m \in \mathbb{N}, 0 \leq m \leq 5\}$
- (f) $\{uu \mid u \in \Sigma^*, |u| = 2\}$
- (g) $\{u0v \mid u \in \Sigma^*, v \in \Sigma^*, |uv| = 2\}$
- (h) $\{uvw \mid u \in \Sigma, v \in \Sigma, w \in \Sigma, w \text{ is the xor of } u \text{ and } v\}$

Solution

- (a) 1, 2, 3, 4, 6
- (b) 2
- (c) (1, 2), (1, 4), (1, 6), (2, 2), (2, 4), (2, 6), (3, 2), (3, 4), (3, 6)
- (d) 1, 3
- (e) 0, 2, 4, 6, 8, 10
- (f) 0000, 0101, 1010, 1111
- (g) 001, 100, 101, 000, 010, 011, 110
- (h) 000, 011, 101, 110

Recall the grammar from the notes for Boolean expressions.

$$\begin{aligned} B &::= F \mid F \parallel B \\ F &::= L \mid L \&\& F \\ L &::= \text{true} \mid \text{false} \mid (B) \end{aligned}$$

* 2. Give derivations for the following strings which are in the language of Boolean expressions.

- (a) true && false
- (b) true || false && true
- (c) true && (false || true)

Solution

There are different possible derivations depending on which non-terminals are chosen to be replaced. I will generally replace leftmost non-terminals at each step.

(a)

$$\begin{aligned} B &\rightarrow F \\ &\rightarrow L \&\& F \\ &\rightarrow \text{true} \&\& F \\ &\rightarrow \text{true} \&\& L \\ &\rightarrow \text{true} \&\& \text{false} \end{aligned}$$

(b)

$$\begin{aligned} B &\rightarrow F \parallel B \\ &\rightarrow L \parallel B \\ &\rightarrow \text{true} \parallel B \\ &\rightarrow \text{true} \parallel F \\ &\rightarrow \text{true} \parallel L \&\& F \\ &\rightarrow \text{true} \parallel \text{false} \&\& F \\ &\rightarrow \text{true} \parallel \text{false} \&\& L \\ &\rightarrow \text{true} \parallel \text{false} \&\& \text{true} \end{aligned}$$

(c)

$$\begin{aligned} B &\rightarrow F \\ &\rightarrow L \ \&\& \ F \\ &\rightarrow \text{true} \ \&\& \ F \\ &\rightarrow \text{true} \ \&\& \ L \\ &\rightarrow \text{true} \ \&\& \ (B) \\ &\rightarrow \text{true} \ \&\& \ (F \parallel B) \\ &\rightarrow \text{true} \ \&\& \ (L \parallel B) \\ &\rightarrow \text{true} \ \&\& \ (\text{false} \parallel B) \\ &\rightarrow \text{true} \ \&\& \ (\text{false} \parallel F) \\ &\rightarrow \text{true} \ \&\& \ (\text{false} \parallel L) \\ &\rightarrow \text{true} \ \&\& \ (\text{false} \parallel \text{true}) \end{aligned}$$

* 3. Give three distinct derivations for the string `true || false`.

Solution

(a)

$$\begin{aligned} B &\rightarrow F \parallel B \\ &\rightarrow L \parallel B \\ &\rightarrow \text{true} \parallel B \\ &\rightarrow \text{true} \parallel F \\ &\rightarrow \text{true} \parallel L \\ &\rightarrow \text{true} \parallel \text{false} \end{aligned}$$

(b)

$$\begin{aligned} B &\rightarrow F \parallel B \\ &\rightarrow F \parallel F \\ &\rightarrow L \parallel F \\ &\rightarrow \text{true} \parallel F \\ &\rightarrow \text{true} \parallel L \\ &\rightarrow \text{true} \parallel \text{false} \end{aligned}$$

(c)

$$\begin{aligned} B &\rightarrow F \parallel B \\ &\rightarrow F \parallel F \\ &\rightarrow F \parallel L \\ &\rightarrow F \parallel \text{false} \\ &\rightarrow L \parallel \text{false} \\ &\rightarrow \text{true} \parallel \text{false} \end{aligned}$$

$$\begin{aligned}
Cmd &::= \$ \\
&| Exp \$ \\
&| \text{def ident } (ExpList) = Exp \$ \\
Exp &::= \text{var} \\
&| \text{ident } (ExpList) \\
ExpList &::= \epsilon \\
&| Exp ExpList' \\
ExpList' &::= \epsilon \\
&| , Exp ExpList'
\end{aligned}$$

Figure 1: The Microbrew language grammar.

- * 4. Figure 1 gives the production rules for the Microbrew language, which we shall discuss further in Week 3. The grammar has start symbol *Cmd*, and is built over eight terminal symbols:

var ident () , def = \$

Intuitively, the terminal symbol *ident* represents any valid identifier, and *var* represents any valid variable name. The terminal symbol *\$* is used as an explicit end-of-input marker.

Which of the following strings are derivable? You need not give a derivation.

- (a) *ident*(*var*) \$
- (b) *ident*(*ident*()) \$
- (c) *ident*() \$
- (d) *ident*(*ident*, *ident*) \$
- (e) *ident*(*var*) = *ident*(*var*) \$
- (f) *def var* = *ident*() \$
- (g) *def ident*(*var*) = *var* \$
- (h) *def ident*() = *var* \$
- (i) (*ident*()) \$
- (j) (*def ident*() = *var*) \$

Solution

- (a) yes
- (b) yes

- (c) yes
- (d) no
- (e) no
- (f) no
- (g) yes
- (h) yes
- (i) no
- (j) no

** 5. Consider the following two grammars over the terminal symbols $\{a, b\}$, each of which has start symbol S .

$$S ::= \epsilon \mid aSb \qquad S ::= \epsilon \mid a \mid b \mid aSa \mid bSb$$

In all of the following questions, strings refer to strings over the alphabet $\{a, b\}$.

- (a) Give three strings that are derivable in the left grammar, but not in the right grammar.
- (b) Give three strings that are derivable in the right grammar, but not in the left grammar.
- (c) Give a new context free grammar which derives exactly those strings that are either derivable in the left grammar, or derivable in the right grammar. For example, all six of your strings from parts (a) and (b) should be derivable in the new grammar.
- (d) Give a new context free grammar which derives exactly those strings that are derivable in both the left and right grammars.
- (e) Give a new context free grammar which derives the following language of strings, in which u^R is the reversal of the string u :

$$\{a^n u b^n \mid u \in \{a, b\}^* \wedge n \geq 0 \wedge u = u^R\}$$

(That is, each string in the language consists of some possibly empty prefix of a , followed by some string u which is its own reverse, followed by a suffix of b which is exactly the same length as the prefix of a .)

Solution

- (a) $ab, aabb, aaabbb$
- (b) a, b, aa
- (c)

$$\begin{aligned} S &::= T \mid U \\ T &::= \epsilon \mid aTb \\ U &::= \epsilon \mid a \mid b \mid aUa \mid bUb \end{aligned}$$

(d)

$$S ::= \epsilon$$

(e)

$$\begin{aligned} S &::= T \mid aSb \\ T &::= \epsilon \mid a \mid b \mid aTa \mid bTb \end{aligned}$$

- *** 6. An ϵ -production is a rule of shape $X ::= \epsilon$ (for some nonterminal X). A unit production is a rule of shape $X ::= Y$ (for some non-terminals X and Y). Consider the following grammar G with start symbol S :

$$\begin{aligned} S &::= aSbb \mid T \\ T &::= bTaa \mid S \mid \epsilon \end{aligned}$$

This grammar has two unit productions $S ::= T$ and $T ::= S$, and it has an epsilon production $T ::= \epsilon$.

Give a grammar with *no* unit productions and *no* ϵ -productions that derives exactly the non-empty strings that are already derivable in G (i.e. every *non-empty* string derivable in G should be derivable in the new grammar, and every string derivable in the new grammar should be a non-empty string already derivable in G).

Solution

The following grammar describes the language of all the non-empty strings derivable in G , without using unit or ϵ -productions:

$$S ::= aSbb \mid bSaa \mid abb \mid baa$$

We can obtain it in the following way (which more-or-less follows a general approach to eliminating mutually recursive definitions in programming). First, since $S ::= T$ and $T ::= S$, we can replace the production $T ::= bTaa$ by $T ::= bSaa$ without changing the language of the grammar. The new grammar, call it G' , is:

$$\begin{aligned} S &::= aSbb \mid T \\ T &::= bSaa \mid S \mid \epsilon \end{aligned}$$

The reasoning for why this does not change the language is as follows.

- The new grammar (with this production replaced) cannot derive any strings that were not already derivable in the old grammar, because we could already derive $T \rightarrow bTaa \rightarrow bSaa$ in the old grammar.
- The new grammar (with this production replaced) can derive all the string that were derivable in the old grammar because we can derive $T \rightarrow bSaa \rightarrow bTaa$ in the new grammar.

Therefore, the new grammar and the old grammar derive the same strings. Now, we have eliminated the directly recursive part of the T -rules, we can look to eliminate T altogether from G' . We can replace the production $S ::= T$ in G' by the three productions $S ::= bSaa \mid S \mid \epsilon$. This does not change the language of the grammar because if we use $S ::= T$, then we must then replace T by something, and these are the three possibilities (so we are just removing a step from the derivation). This gives equivalent grammar G'' :

$$\begin{aligned} S &::= aSbb \mid bSaa \mid S \mid \epsilon \\ T &::= bSaa \mid S \mid \epsilon \end{aligned}$$

But note that $S ::= S$ is a useless production which contributes nothing to the language described, so we can remove it. Then note that it is impossible to introduce a T nonterminal when starting from S , so all the T rules are useless in G'' . Hence, we obtain the equivalent grammar G''' :

$$S ::= aSbb \mid bSaa \mid \epsilon$$

Now we still want to remove the ϵ -production. In general this will change the language, because $S \rightarrow \epsilon$ is a valid derivation. However, we want to describe the language $L(G) \setminus \{\epsilon\}$ anyway, so this derivation should be impossible. We just observe that, in a derivation of a string in $L(G''') \setminus \{\epsilon\}$, the production $S ::= \epsilon$ will only be used after one of the other two productions. Thus we can combine $S \rightarrow aSbb \rightarrow abb$ and $S \rightarrow bSaa \rightarrow baa$ into two single rules and get rid of the ϵ -production:

$$S ::= aSbb \mid bSaa \mid abb \mid baa$$